

Mapeamento Fonte × Teste

PropostaService

Classe / Método	Tem teste?	Observação
PropostaSeguro (construtor + AlterarStatus + EstaAprovada)	✅ Sim — 11 casos	Cobertura excelente: valida nome v inválido, valor ≤ 0 , todas as transiç válidas/inválidas, status repetido, e EstaAprovada() (via Assert nos testes de transição)
CriarPropostaUseCase.ExecutarAsync	✅ Sim — 2 casos	2 Caminho feliz + propagação de DomainException Cobre "não encontrada" e transição um teste onde a proposta existe ma inválida (ex. já Rejeitada) — hoje e validada no nível da entidade, nunca orquestração do UseCase, então não que AtualizarAsync não seja o cenário via o UseCase
AlterarStatusPropostaUseCase.ExecutarAsync	⚠️ Parcial — 2 casos	Não existe ConsultarPropostaUseCase Deveria cobrir: retorno de DTO qu existe e retorno de null quando ObterPorIdAsync retorna null
ConsultarPropostaUseCase.ExecutarAsync	❌ Nenhum teste	Não existe ListarPropostasUseCaseTest Deveria cobrir: lista com itens e lis
ListarPropostasUseCase.ExecutarAsync	❌ Nenhum teste	Só é exercitado indiretamente atrav UseCase. O ArgumentException.ThrowIfNullOrEmpty(proposta) nunca é testado
PropostaMapper.ParaDto	❌ Nenhum teste direto	Esperado num projeto só de testes (exigiria integração/InMemory ou — não é obrigatório, mas vale regi
PropostaRepository / PropostaDbContext (Infrastructure)	❌ Nenhum teste	Idem — normalmente coberto por integração, ainda não implementad
PropostasController (Api)	❌ Nenhum teste	

ContratacaoService

Classe / Método	Tem teste?	Observação
ContratarPropostaUseCase.ExecutarAsync	✅ Sim — 4 casos, muito bem feito	Proposta não encontrada, status $\neq$ Aprovada (via [Theory] com 3 valores), já contratada, e caminho feliz com verificação do DTO de retorno
Contratacao (entidade de Domínio)	❌ Nenhum teste dedicado	Este é o gap mais importante. Diferente do PropostaService, não existe um ContratacaoTests.cs em tests/Contratacao.Unit

Classe / Método	Tem teste?	Observação
		Tests/Domain/ — a pasta Domain nem existe na suíte de testes do ContracaoService. A regra <code>if (propostaId == Guid.Empty) throw new DomainException(...)</code> no construtor de Contracao nunca é testada diretamente . Ela só é tocada indiretamente pelo caminho feliz do teste de UseCase (sempre com um <code>Guid.NewGuid()</code> válido)
ContratacaoRepository (Infrastructure)	✗ Nenhum teste	Mesma observação do PropostaRepository Este adaptador faz parsing manual da resposta JSON do PropostaService — é exatamente o tipo de lógica (parsing, tratamento de resposta malformada/404/timeout) que mais se beneficiaria de teste unitário isolado (mockando <code>HttpMessageHandler</code>)
PropostaServiceHttpClient (Infrastructure/ExternalServices)	✗ Nenhum teste	
ContratacoesController (Api)	✗ Nenhum teste	Idem PropostaService

✓ O que fazer para fechar a cobertura

Prioridade, do mais crítico ao mais opcional:

1. **ContratacaoTests.cs** (novo arquivo, Domain/) — testar que `new Contratacao(Guid.Empty)` lança `DomainException`, e que `new Contratacao(propostaIdValido)` preenche `Id`, `PropostaId` e `DataContratacao` corretamente. É a única lacuna que deixa uma regra de negócio **sem nenhuma cobertura**, direta ou indireta.
2. **ConsultarPropostaUseCaseTests.cs** e **ListarPropostasUseCaseTests.cs** — simples, 2 casos cada (achou/não achou; lista cheia/vazia).
3. Um teste adicional em `AlterarStatusPropostaUseCaseTests` para o cenário de transição inválida `through-the-usecase` (garante que `AtualizarAsync` nunca é chamado quando a entidade rejeita a transição).
4. `PropostaMapperTests.cs` — trivial, mas fecha 100% da camada Application.
5. (Opcional, maior esforço) Teste unitário do `PropostaServiceHttpClient` mockando `HttpMessageHandler`, para validar o parsing manual do JSON em cenários de resposta malformada.